

```
In [20]: # A/B Testing Campaign Analysis: Comprehensive Script
```

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
import scipy.stats as st
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.cluster import KMeans

import warnings
warnings.filterwarnings('ignore')

# Load the datasets
control_df = pd.read_csv('control_group.csv', delimiter=';')
test_df = pd.read_csv('test_group.csv', delimiter=';')

# Drop missing values
control_df_clean = control_df.dropna()
test_df_clean = test_df.dropna()

# Convert numerical columns to float
cols_to_convert = control_df_clean.columns.drop(['Campaign Name', 'Date'])
control_df_clean[cols_to_convert] = control_df_clean[cols_to_convert].astype(float)
test_df_clean[cols_to_convert] = test_df_clean[cols_to_convert].astype(float)

# Add metrics
for df in [control_df_clean, test_df_clean]:
    df['Conversion Rate'] = df['# of Purchase'] / df['# of Website Clicks']
    df['ROAS'] = df['# of Purchase'] / df['Spend [USD]']

# Combine and label
test_df_clean['Group'] = 'Test'
control_df_clean['Group'] = 'Control'
combined_df = pd.concat([control_df_clean, test_df_clean], ignore_index=True)
combined_df['Date'] = pd.to_datetime(combined_df['Date'], format='%d.%m.%Y')
combined_df = combined_df.sort_values('Date')

# Summary Statistics and T-Tests
summary_stats = combined_df.groupby('Group')[['Conversion Rate', 'ROAS']].agg(['mean', 'std', 'count'])
from scipy.stats import ttest_ind
t_conv, p_conv = ttest_ind(control_df_clean['Conversion Rate'], test_df_clean['Conversion Rate'], equal_var=False)
t_roas, p_roas = ttest_ind(control_df_clean['ROAS'], test_df_clean['ROAS'], equal_var=False)

# Confidence Intervals and Visual Comparison
metrics = ['Conversion Rate', 'ROAS']
def mean_ci(series, confidence=0.95):
    n = len(series)
    mean = np.mean(series)
    stderr = st.sem(series)
    margin = stderr * st.t.ppf((1 + confidence) / 2, n - 1)
    return mean, mean - margin, mean + margin
results = []
for metric in metrics:
    for group, data in combined_df.groupby('Group'):
        mean, lower, upper = mean_ci(data[metric])
        results.append({'Metric': metric, 'Group': group, 'Mean': mean, 'CI Lower': lower, 'CI Upper': upper})
results_df = pd.DataFrame(results)
fig, axes = plt.subplots(1, 2, figsize=(14, 6))
for ax, metric in zip(axes, metrics):
    data = results_df[results_df['Metric'] == metric]
    ax.bar(data['Group'], data['Mean'], yerr=[data['Mean'] - data['CI Lower'], data['CI Upper'] - data['Mean']], capsize=10)
    ax.set_title(f'{metric} by Group with 95% CI')
    ax.grid(True)
plt.tight_layout()
plt.show()

# Time Series Performance
fig, axes = plt.subplots(2, 1, figsize=(14, 10), sharex=True)
sns.lineplot(data=combined_df, x='Date', y='Conversion Rate', hue='Group', marker='o', ax=axes[0])
sns.lineplot(data=combined_df, x='Date', y='ROAS', hue='Group', marker='o', ax=axes[1])
axes[0].set_title('Daily Conversion Rate by Group')
axes[1].set_title('Daily ROAS by Group')
axes[0].grid(True)
axes[1].grid(True)
plt.tight_layout()
plt.show()

# Cumulative Performance and Efficiency
combined_df['Cumulative Spend'] = combined_df.groupby('Group')['Spend [USD]'].cumsum()
combined_df['Cumulative Purchases'] = combined_df.groupby('Group')['# of Purchase'].cumsum()
combined_df['Cumulative Website Clicks'] = combined_df.groupby('Group')['# of Website Clicks'].cumsum()
combined_df['Purchase per Impression'] = combined_df['# of Purchase'] / combined_df['# of Impressions']
combined_df['Purchase per Reach'] = combined_df['# of Purchase'] / combined_df['Reach']
fig, axes = plt.subplots(3, 1, figsize=(14, 14), sharex=True)
sns.lineplot(data=combined_df, x='Date', y='Cumulative Spend', hue='Group', marker='o', ax=axes[0])
sns.lineplot(data=combined_df, x='Date', y='Cumulative Purchases', hue='Group', marker='o', ax=axes[1])
sns.lineplot(data=combined_df, x='Date', y='Cumulative Website Clicks', hue='Group', marker='o', ax=axes[2])
axes[0].set_title('Cumulative Spend Over Time')
axes[1].set_title('Cumulative Purchases Over Time')
axes[2].set_title('Cumulative Website Clicks Over Time')
plt.tight_layout()
plt.show()

# Purchase Efficiency over Time
fig, axes = plt.subplots(2, 1, figsize=(14, 10), sharex=True)
sns.lineplot(data=combined_df, x='Date', y='Purchase per Impression', hue='Group', marker='o', ax=axes[0])
sns.lineplot(data=combined_df, x='Date', y='Purchase per Reach', hue='Group', marker='o', ax=axes[1])
axes[0].set_title('Purchase per Impression Over Time')
axes[1].set_title('Purchase per Reach Over Time')
plt.tight_layout()
plt.show()

# Funnel Metrics
combined_df['CTR'] = combined_df['# of Website Clicks'] / combined_df['# of Impressions']
combined_df['Search Rate'] = combined_df['# of Searches'] / combined_df['# of Website Clicks']
combined_df['Content View Rate'] = combined_df['# of View Content'] / combined_df['# of Searches']
combined_df['Add-to-Cart Rate'] = combined_df['# of Add to Cart'] / combined_df['# of View Content']
combined_df['Final Purchase Rate'] = combined_df['# of Purchase'] / combined_df['# of Add to Cart']

# Daily Consistency
daily_std = combined_df.groupby('Group')[['Conversion Rate', 'CTR', 'ROAS']].std()

# Spend vs Conversion Correlation
spend_corr = combined_df[['Spend [USD]', '# of Website Clicks', '# of Purchase', 'ROAS']].corr()

# Composite Index
scaler = StandardScaler()
combined_df[['CTR', 'Conversion Rate', 'ROAS']] = scaler.fit_transform(combined_df[['CTR', 'Conversion Rate', 'ROAS']])
combined_df['Performance Index'] = combined_df[['CTR', 'Conversion Rate', 'ROAS']].mean(axis=1)

# Predictive Modeling
X = combined_df[['Spend [USD]', '# of Impressions', 'Reach', '# of Website Clicks', '# of Searches', '# of View Content', '# of Add to Cart']]
y = combined_df['# of Purchase']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```

model = LinearRegression().fit(X_train, y_train)
feature_importance = pd.Series(model.coef_, index=X.columns).sort_values(ascending=False)

# Clustering Performance Days
cluster_features = combined_df[['# of Website Clicks', '# of Purchase', 'ROAS', 'Conversion Rate']]
cluster_data = scaler.fit_transform(cluster_features)
combined_df['Performance Cluster'] = KMeans(n_clusters=3, random_state=42).fit_predict(cluster_data)

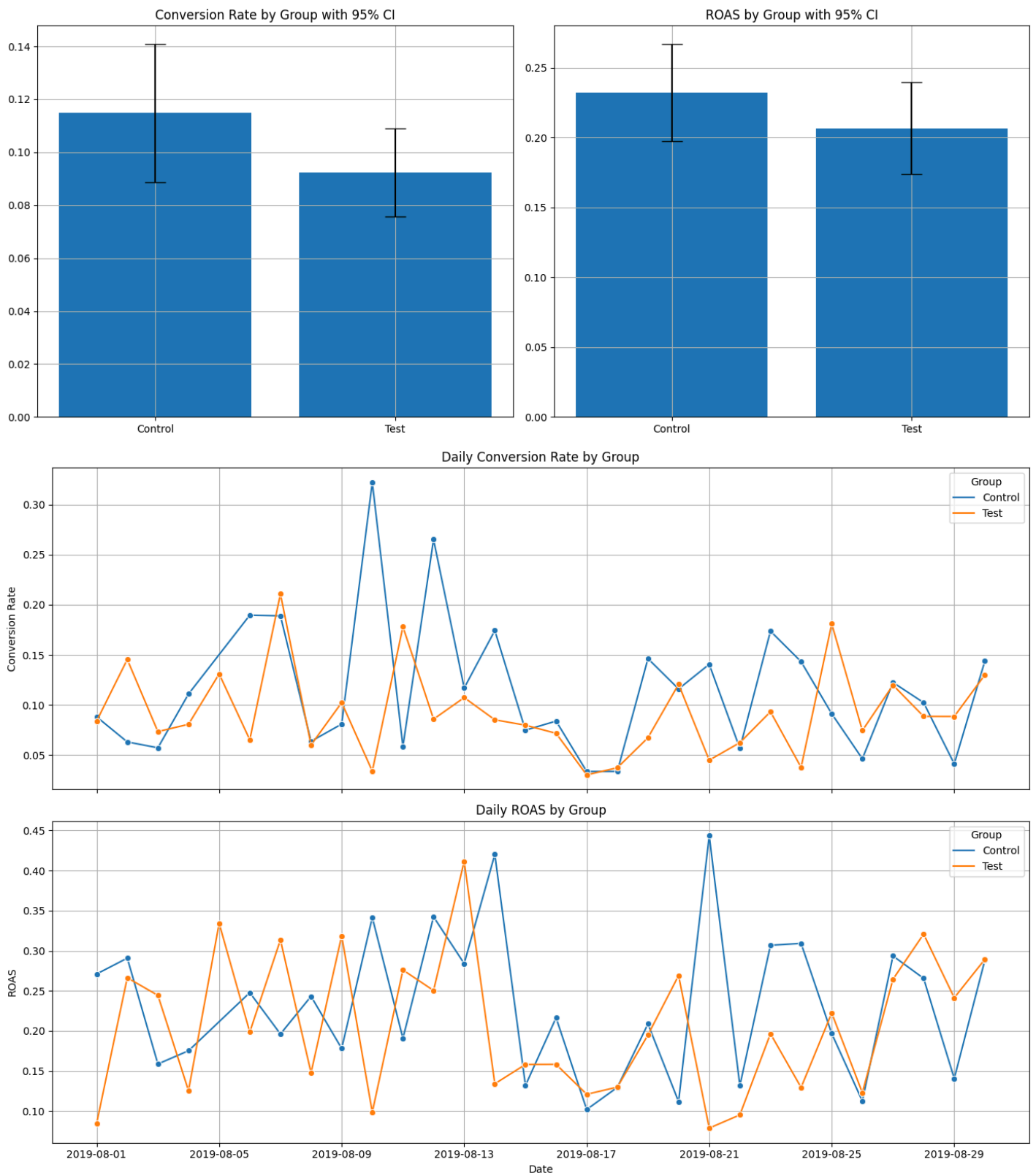
# Rolling Averages for CTR and ROAS
combined_df['Rolling CTR'] = combined_df.groupby('Group')['CTR'].transform(lambda x: x.rolling(5, min_periods=1).mean())
combined_df['Rolling ROAS'] = combined_df.groupby('Group')['ROAS'].transform(lambda x: x.rolling(5, min_periods=1).mean())

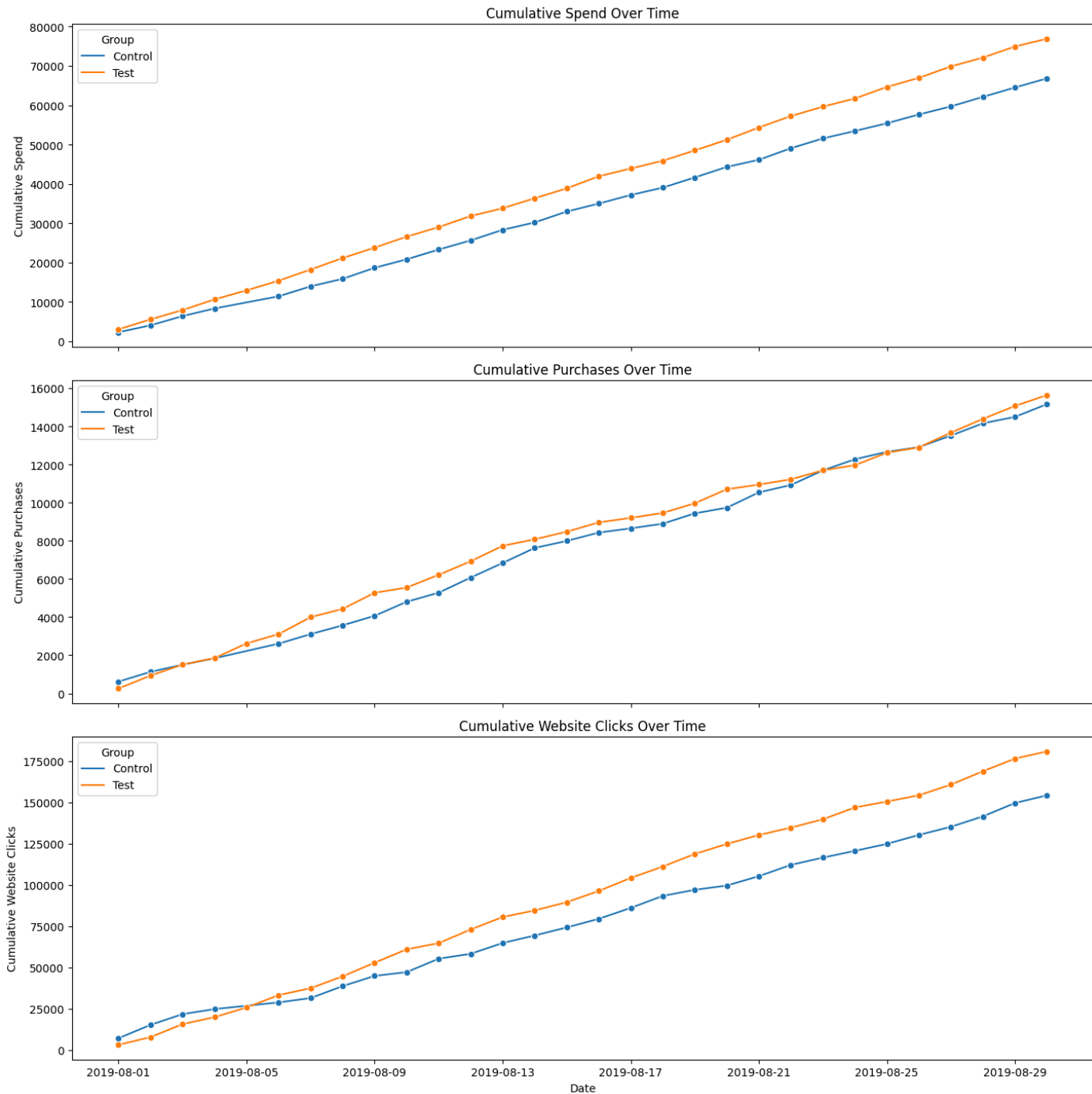
# Final Visualizations
funnel_means = combined_df.groupby('Group')[['CTR', 'Search Rate', 'Content View Rate', 'Add-to-Cart Rate', 'Final Purchase Rate']].mean().T
funnel_means.plot(kind='bar', figsize=(12, 6), title="Funnel Efficiency by Group")
plt.ylabel("Rate")
plt.grid(True)
plt.tight_layout()
plt.show()

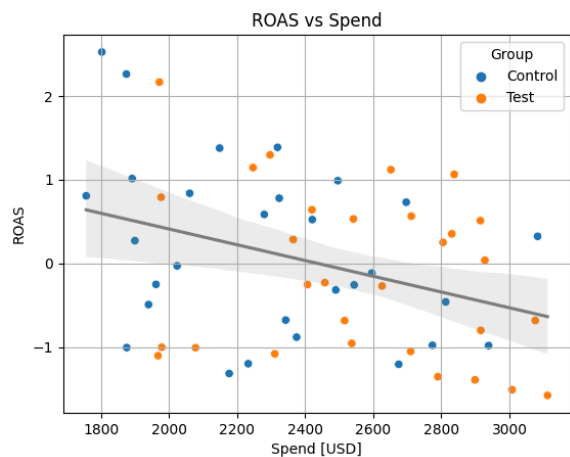
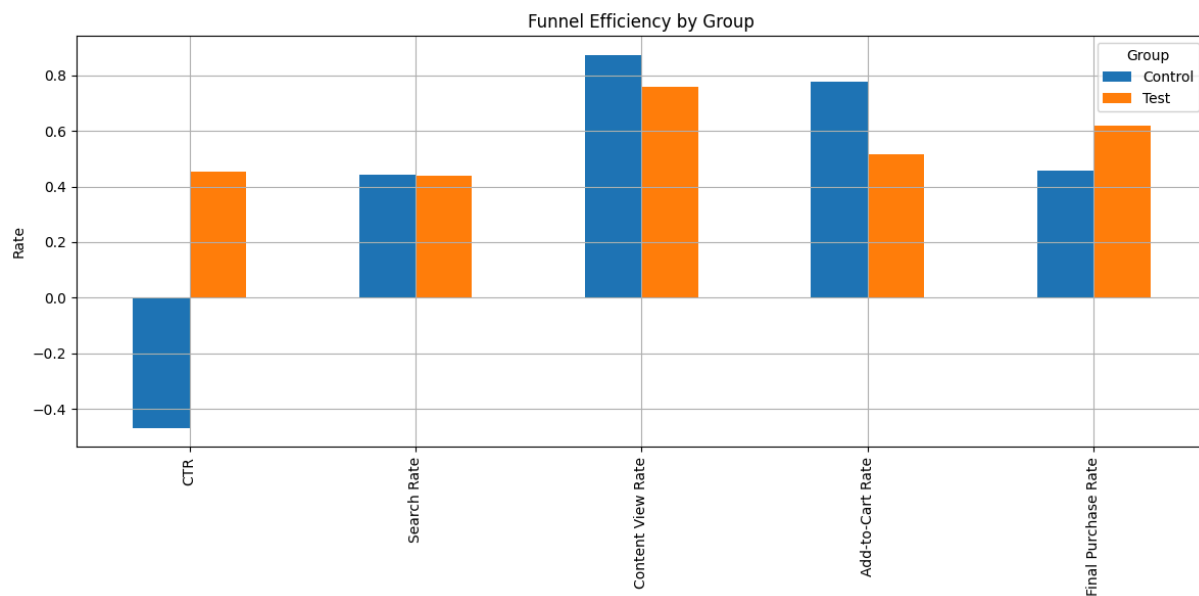
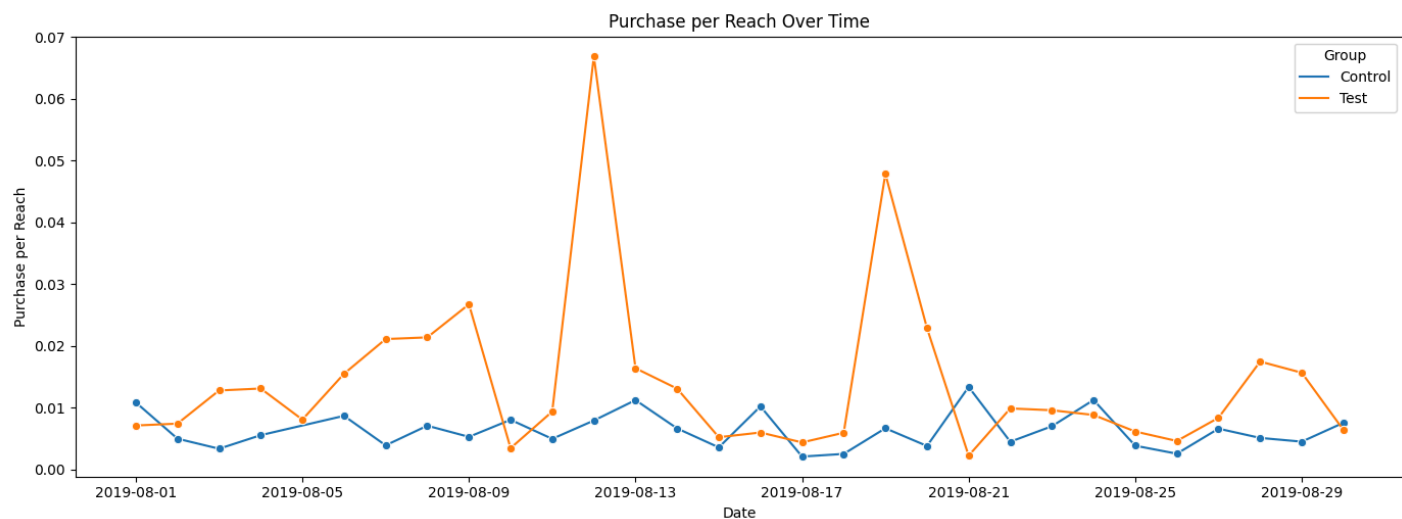
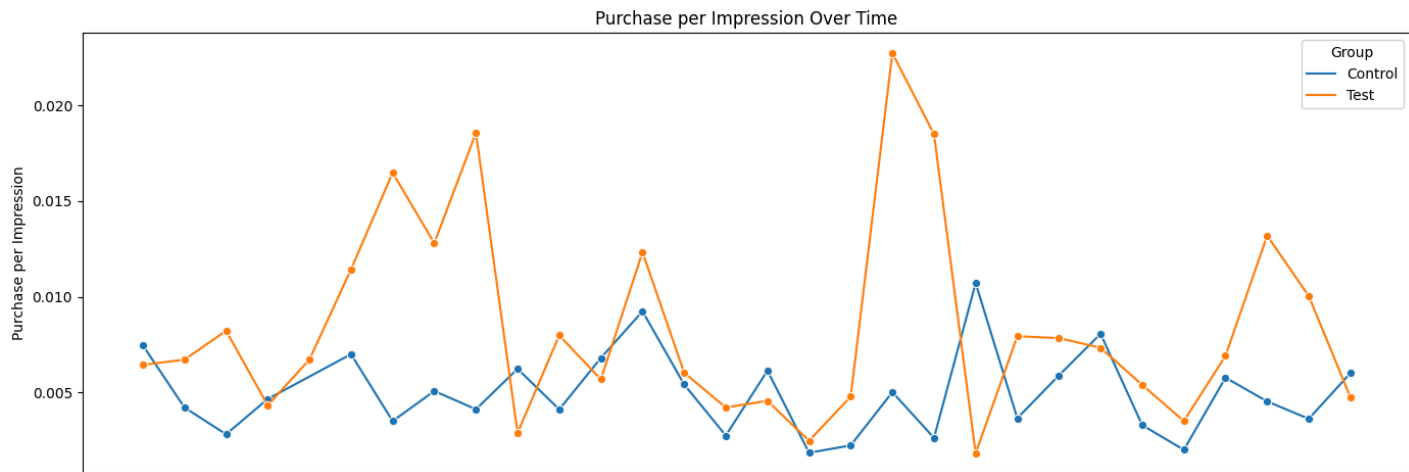
sns.scatterplot(data=combined_df, x='Spend [USD]', y='ROAS', hue='Group')
sns.regplot(data=combined_df, x='Spend [USD]', y='ROAS', scatter=False, color='gray')
plt.title("ROAS vs Spend")
plt.grid(True)
plt.show()

sns.scatterplot(data=combined_df, x='Conversion Rate', y='ROAS', hue='Performance Cluster', style='Group')
plt.title("Clusters of Campaign Days")
plt.grid(True)
plt.show()

```







Clusters of Campaign Days

